Assembly Language for x86 Processors
7th Edition, Global Edition

Kip R. Irvine

Chapter 8: Advanced Procedures

Slides prepared by the author.

(c) Pearson Education, 2015. All rights reserved. You may modify and copy this slide show for your personal use, or for use in the classroom, as long as this copyright statement, the author's name, and the title are not changed.

PROC Directive (1 of 2)

- The PROC directive declares a procedure with an optional list of named parameters.
- Syntax:

  label PROC paramList
- *paramList* is a list of parameters separated by commas. Each parameter has the following syntax:

  paramName **:** *type*

type must either be one of the standard ASM types (BYTE, SBYTE, WORD, etc.), or it can be a pointer to one of these types.

Not in 64-bit mode!

PROC Directive (2 of 2)

- Alternate format permits parameter list to be on one or more separate lines:

  label PROC**,** comma required

  paramList

- The parameters can be on the same line . . .

  param-1:type-1, param-2:type-2, . . ., param-n:type-n

- Or they can be on separate lines:

  param-1:type-1,

  param-2:type-2,

  . . .,

  param-n:type-n

AddTwo Procedure

- The AddTwo procedure receives two integers and returns their sum in EAX.

```
AddTwo PROC,
    val1:DWORD, val2:DWORD

    mov eax,val1
    add eax,val2

    ret
AddTwo ENDP
```

INVOKE Directive

- In 32-bit mode, the INVOKE directive is a powerful replacement for Intel's CALL instruction that lets you pass multiple arguments
- Syntax:

  INVOKE *procedureName* [, *argumentList*]

- *ArgumentList* is an optional comma-delimited list of procedure arguments
- Arguments can be:
  - -immediate values and integer expressions
  - -variable names
  - -address and ADDR expressions
  - -register names

PROTO Directive

- Creates a procedure prototype
- Syntax:
  - *label* PROTO *paramList*
- Parameter list not permitted in 64-bit mode
- Every procedure called by the INVOKE directive must have a prototype
- A complete procedure definition can also serve as its own prototype

PROTO Directive

- Standard configuration: PROTO appears at top of the program listing, INVOKE appears in the code segment, and the procedure implementation occurs later in the program:

```
MySub PROTO          ; procedure prototype

.code
INVOKE MySub         ; procedure call


MySub PROC           ; procedure implementation
    .
    .
MySub ENDP
```

PROTO Example

- Prototype for the ArraySum procedure, showing its parameter list:

```
ArraySum PROTO,
    ptrArray:PTR DWORD,     ; points to the array
    szArray:DWORD           ; array size
```

Stack Frame

- Also known as an *activation record*
- Area of the stack set aside for a procedure's return address, passed parameters, saved registers, and local variables
- Created by the following steps:
  - Calling program pushes arguments on the stack and calls the procedure.
  - The called procedure pushes EBP on the stack, and sets EBP to ESP.
  - If local variables are needed, a constant is subtracted from ESP to make room on the stack.

Stack Parameters

- More convenient than register parameters
- Two possible ways of calling a function. Which is easier?

```
pushad
mov esi,OFFSET array
mov ecx,LENGTHOF array
mov ebx,TYPE array
call DumpMem
popad
```

```
push TYPE array
push LENGTHOF array
push OFFSET array
call DumpMem
```

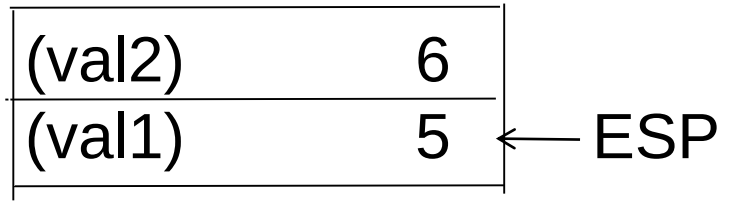
Passing Arguments by Value

- Push argument values on stack
  - (Use only 32-bit values in protected mode to keep the stack aligned)
- Call the called-procedure
- Accept a return value in EAX, if any
- Remove arguments from the stack if the called-procedure did not remove them

Example

```
.data
val1  DWORD 5
val2  DWORD 6

.code
push val2
push val1
Call AddTwo
```



Stack prior to CALL

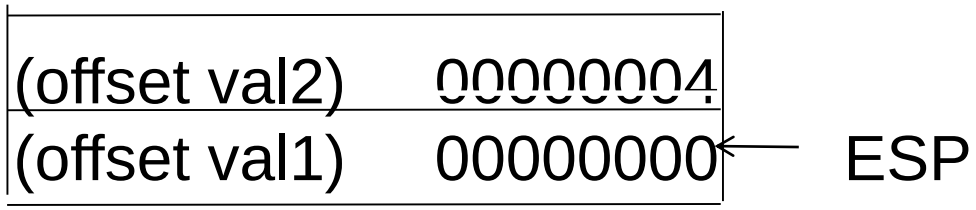
Passing by Reference

- Push the offsets of arguments on the stack
- Call the procedure
- Accept a return value in EAX, if any
- Remove arguments from the stack if the called procedure did not remove them

Example

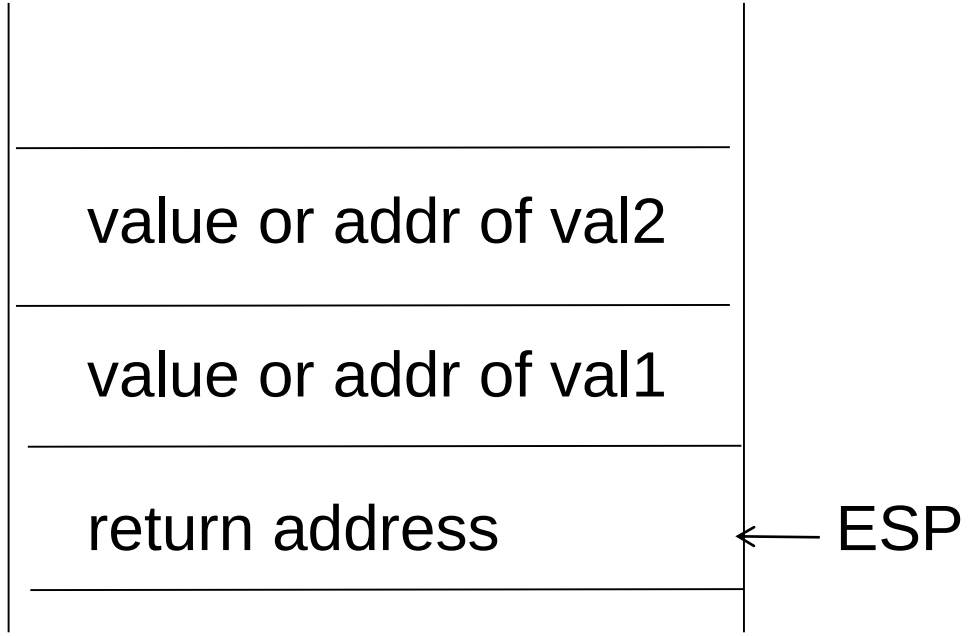
```
.data
val1  DWORD 5
val2  DWORD 6

.code
push OFFSET val2
push OFFSET val1
Call Swap
```



Stack prior to CALL

Stack after the CALL



RET Instruction

- *Return from subroutine*
- Pops stack into the instruction pointer (EIP or IP). Control transfers to the target address.
- Syntax:

  RET

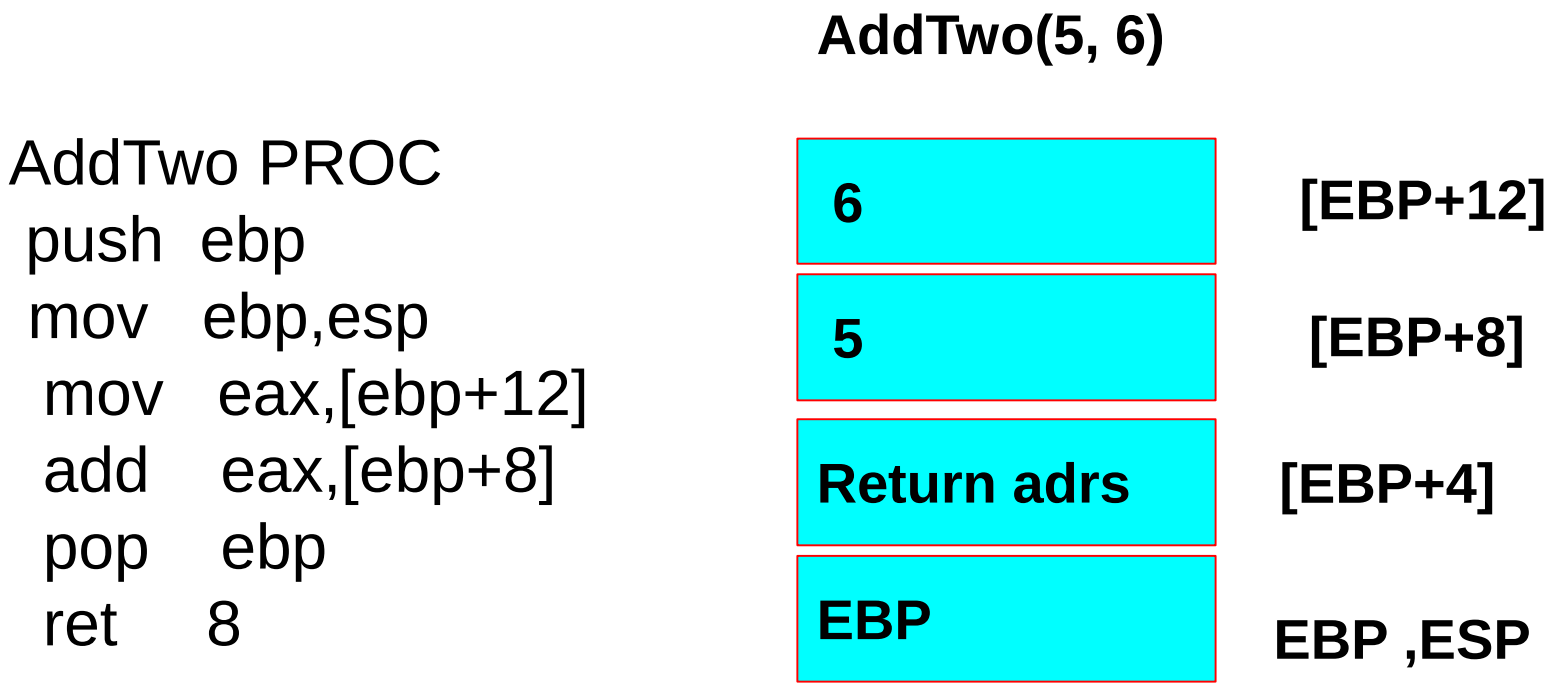
  RET *n*
- Optional operand *n* causes *n* bytes to be added to the stack pointer after EIP (or IP) is assigned a value

Accessing Stack Parameters (C/C++)

- C and C++ functions access stack parameters using constant offsets from EBP.
  - Example: [ebp + 8]
- EBP is called the base pointer or frame pointer because it holds the base address of the stack frame.
- EBP does not change value during the function.
- EBP must be restored to its original value when a function returns.

Base-Offset Addressing

```
AddTwo PROC
 push  ebp
 mov   ebp,esp
  mov   eax,[ebp+12]
  add    eax,[ebp+8]
  pop    ebp
  ret     8
```



Passing 8-bit and 16-bit Arguments

- Cannot push 8-bit values on stack
- Pushing 16-bit operand may cause page fault or ESP alignment problem
- Expand smaller arguments into 32-bit values using MOVZX

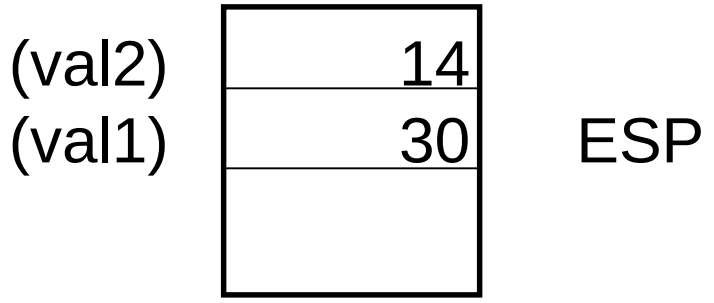
```
.data
charVal BYTE 'x'
.code
  movzx eax,charVal
  push  eax
  call  Uppercase
```

Your turn

- Create a procedure named Difference that subtracts the second argument(Val2) from the first one(Val1).

  Val1 – Val2
- Following is a sample call:

```
push 14                    ; Val2
push 30                    ; Val1
call Difference            ; EAX = 16
```



Local Variables

- Only statements within subroutine can view or modify local variables
- Storage used by local variables is released when subroutine ends
- local variable name can have the same name as a local variable in another function without creating a name clash
- Essential when writing recursive procedures, as well as procedures executed by multiple execution threads

Local Variables

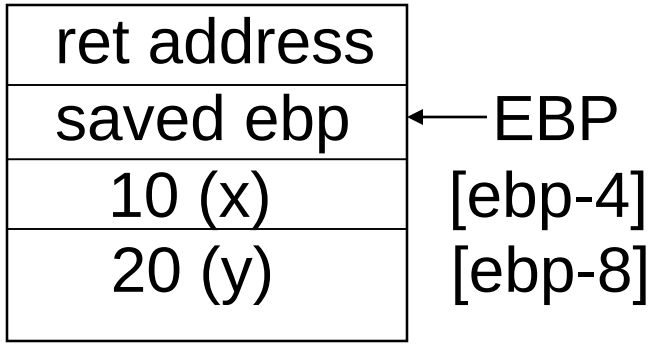
- To explicitly create local variables, subtract their total size from ESP.
- The following example creates and initializes two 32-bit local variables (we'll call them locA and locB):

```
MySub PROC
    push ebp
    mov  ebp,esp
    add  esp,-8
    mov  [ebp-4],123456h          ; locA
    mov  [ebp-8],0                ; locB
    .
    .
```

Creating LOCAL Variables

Example - create two DWORD local variables:

Say: int x=10, y=20;




```
MySub PROC
      push  ebp
      mov   ebp,esp
      sub   esp,8          ;create 2 DWORD variables

      mov   DWORD PTR [ebp-4],10 ; initialize x=10
      mov   DWORD PTR [ebp-8],20 ; initialize y=20
```

LEA Instruction

- The LEA instruction returns offsets
  - OFFSET operator can only return constant offsets.
- LEA is required when obtaining the offset of a stack parameter or local variable. For example:

```
CopyString PROC, count:DWORD
    LOCAL temp[20]:BYTE

    mov edi,OFFSET count        ; invalid operand
    mov esi,OFFSET temp         ; invalid operand

                                ; which instruction should be used?
```

LEA Example

- Suppose you have a Local variable at [ebp-8]
- And you need the address of that local variable in ESI
- You cannot use this:

```
mov esi, OFFSET [ebp-8]        ; error
```

- Use this instead:

```
lea esi,[ebp-8]
```

ENTER Instruction

- ENTER instruction creates stack frame for a called procedure
  - pushes EBP on the stack
  - sets EBP to the base of the stack frame
  - reserves space for local variables
  - Example:

```
MySub PROC
  enter 8,0
```

  - Equivalent to:

```
MySub PROC
  push ebp
  mov ebp,esp
  sub esp,8
```

LEAVE Instruction

Terminates the stack frame for a procedure.

```
MySub PROC
        enter 8,0
        ...
        ...
        ...
        leave
        ret
MySub ENDP
```

Equivalent operations

```
push   ebp
mov    ebp,esp
sub    esp,8      ; 2 local DWORDs
```

```
mov    esp,ebp  ; free local space
pop    ebp
```

LOCAL Directive

- The LOCAL directive declares a list of local variables
  - immediately follows the PROC directive
  - each variable is assigned a type
- Syntax:

  LOCAL *varlist*

Example:

```
MySub PROC
    LOCAL var1:BYTE, var2:WORD, var3:SDWORD
```

LOCAL Example (1 of 2)

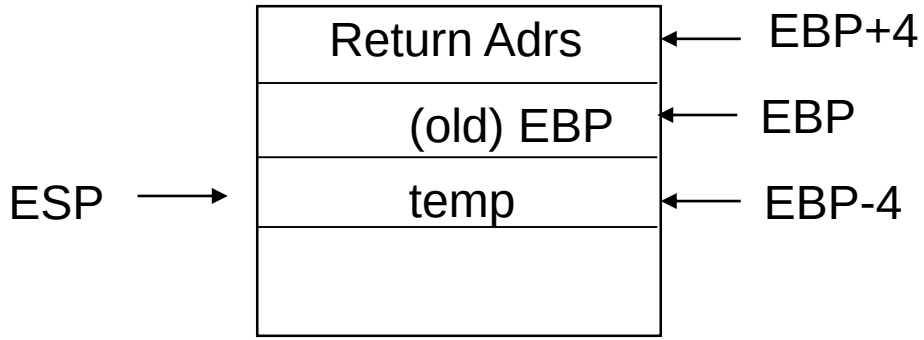
```
Example1 PROC
   LOCAL temp:DWORD
   . . .
   ret
Example1 ENDP
```

MASM generates the following code:

```
Example1 PROC
   push ebp
   mov  ebp,esp
   add  esp, 0FFFFFFFCh       ; add -4 to ESP
   . . .
   mov  esp,ebp
   pop  ebp
   ret
Example1 ENDP
```

LOCAL Example (2 of 2)

Diagram of the stack frame for the `Example1` procedure:



INVOKE Directive

- In 32-bit mode, the INVOKE directive is a powerful replacement for Intel's CALL instruction that lets you pass multiple arguments
- Syntax:

  `INVOKE` ***`procedureName`*** **`[,`** ***`argumentList`*** **`]`**

- *ArgumentList* is an optional comma-delimited list of procedure arguments
- Arguments can be:
  - immediate values and integer expressions
  - variable names
  - address and ADDR expressions
  - register names

Not in 64-bit mode!

INVOKE Examples

```
.data
byteVal BYTE 10
wordVal WORD 1000h
.code
   ; direct operands:
   INVOKE Sub1,byteVal,wordVal

   ; address of variable:
   INVOKE Sub2,ADDR byteVal

   ; register name, integer expression:
   INVOKE Sub3,eax,(10 * 20)

   ; address expression (indirect operand):
   INVOKE Sub4,[ebx]
```

Not in 64-bit mode!

ADDR Operator

- Available in 32-bit mode
- Can be used to pass a pointer argument when calling a procedure using INVOKE
- Returns a near or far pointer to a variable, depending on which memory model your program uses:
  - Small model: returns 16-bit offset
  - Large model: returns 32-bit segment/offset
  - Flat model: returns 32-bit offset
- Simple example:

```
.data
myWord WORD ?
.code
INVOKE mySub,ADDR myWord
```

INVOKE and ADDR

```
.data
Array DWORD 20 DUP(?)
.code
…
INVOKE Swap, ADDR Arrary, ADDR [Array+4]
```

Corresponding Code



INVOKE and ADDR

```
.data
Array DWORD 20 DUP(?)
.code
…
INVOKE Swap, ADDR Arrary, ADDR [Array+4]
```

Corresponding Code

```
push OFFSET Array+4
push OFFSET Array
call Swap
```

Not in 64-bit mode!

PROC Directive (1 of 2)

- The PROC directive declares a procedure with an optional list of named parameters.
- Syntax:

  label PROC paramList
- *paramList* is a list of parameters separated by commas. Each parameter has the following syntax:

  paramName **:** *type*

type must either be one of the standard ASM types (BYTE, SBYTE, WORD, etc.), or it can be a pointer to one of these types.

PROC Directive (2 of 2)

- Alternate format permits parameter list to be on one or more separate lines:

  label PROC**,** comma required

  paramList

- The parameters can be on the same line . . .

  param-1:type-1, param-2:type-2, . . ., param-n:type-n

- Or they can be on separate lines:

  param-1:type-1,

  param-2:type-2,

  . . .,

  param-n:type-n

AddTwo Procedure

- The AddTwo procedure receives two integers and returns their sum in EAX.

```
AddTwo PROC,
    val1:DWORD, val2:DWORD

    mov eax,val1
    add eax,val2

    ret
AddTwo ENDP
```

AddTwo Procedure

The assembly language generated by MASM when assembling AddTwo shows how the parameter names are translated into offsets from EBP.

```
AddTwo PROC
  push ebp
  mov ebp, esp
  mov eax, dword ptr [ebp+8]
  add eax, dword ptr  [ebp+0ch]
  leave
  ret 8
AddTwo ENDP
```

PROC Examples

```
Swap PROC,
   pValX:PTR DWORD,
   pValY:PTR DWORD
   . . .
Swap ENDP


ReadFile PROC,
   pBuffer:PTR BYTE
   LOCAL fileHandle:DWORD
   . . .
ReadFile ENDP
```

PROC Examples

FillArray receives a pointer to an array of bytes, a single byte fill value that will be copied to each element of the array, and the size of the array.

```
FillArray PROC,
   pArray:PTR BYTE, fillVal:BYTE
   arraySize:DWORD

   mov ecx,arraySize
   mov esi,pArray
   mov al,fillVal
L1:mov [esi],al
   inc esi
   loop L1
   ret
FillArray ENDP
```

PROC Examples

FillArray receives a pointer to an array of bytes, a single byte fill value that will be copied to each element of the array, and the size of the array.

```
FillArray PROC,
   pArray:PTR BYTE, fillVal:BYTE
   arraySize:DWORD

   mov ecx,arraySize
   mov esi,pArray
   mov al,fillVal
L1:mov [esi],al
   inc esi
   loop L1
   ret
FillArray ENDP
```

```
.data
Ex1Array sdword 10,-20,30,50,40,-2
.code
main proc
mov eax, Ex1Array+12
mov ebx, [ Ex1Array+12 ]
mov ecx, offset Ex1Array+12
lea edx, Ex1Array+12
call DumpRegs
```

……



```
.data
Ex1Array sdword 10,-20,30,50,40,-2          ; 50  = 16*3 +2 = 0000 0032h
.code
main proc
mov eax, Ex1Array+12
mov ebx, [ Ex1Array+12 ]
mov ecx, offset Ex1Array+12
lea edx, Ex1Array+12
call DumpRegs
```

……

```
EAX=00000032  EBX=00000032  ECX=00404010  EDX=00404010
ESI=00000000  EDI=00000000  EBP=0018FF90  ESP=0018FF88
EIP=0040101B  EFL=00000246  CF=0  SF=0  ZF=1  OF=0  AF=0  PF=1
```

```
TestStack PROC,
        arg1:WORD,
        arg2:BYTE
        LOCAL temp[10]:BYTE
    call DumpRegs ;觀察esp
    mov ax,arg1
    push eax
    call DumpRegs ;觀察esp
    push ax
    call DumpRegs ;觀察esp
    push WORD PTR 10
    call DumpRegs ;觀察esp
    push 10
    call DumpRegs ;觀察esp
    ret 8
TestStack endp ……
```

Before calling TestStack

ESP=0019FF74

ESP=0019FF58 EBP=0019FF64

ESP=0019FF54

ESP=0019FF52

ESP=0019FF50

ESP=0019FF4C

Creating Multimodule Programs

- A multimodule program is a program whose source code has been divided up into separate ASM files.
- Each ASM file (module) is assembled into a separate OBJ file.
- All OBJ files belonging to the same program are linked using the link utility into a single EXE file.

Advantages

- Large programs are easier to write, maintain, and debug when divided into separate source code modules.
- When changing a line of code, only its enclosing module needs to be assembled again. Linking assembled modules requires little time.
- A module can be a container for logically related code and data
  - encapsulation: procedures and variables are automatically hidden in a module unless you declare them public

Creating a Multimodule Program

- Here are some basic steps to follow when creating a multimodule program:
  - Create the main module
  - Create a separate source code module for each procedure or set of related procedures
  - Create an include file that contains procedure prototypes for external procedures (ones that are called between modules)
  - Use the INCLUDE directive to make your procedure prototypes available to each module

INCLUDE File

The sum.inc file contains prototypes for external functions that are not in the Irvine32 library:

```
INCLUDE Irvine32.inc

PromptForIntegers PROTO,
   ptrPrompt:PTR BYTE,        ; prompt string
   ptrArray:PTR DWORD,        ; points to the array
   arraySize:DWORD            ; size of the array

ArraySum PROTO,
   ptrArray:PTR DWORD,        ; points to the array
   count:DWORD                ; size of the array

DisplaySum PROTO,
   ptrPrompt:PTR BYTE,        ; prompt string
   theSum:DWORD               ; sum of the array
```

Homework 3

- Write a procedure named CountMatches that receives points to two arrays of signed doublewords, and a third parameter that indicates the length of the two arrays.
- For each element **xi in the first array, if the corresponding yi** in the second array is equal, increment a counter. At the end, return a count of the number of matching array elements in EAX.
- Write a test program that calls CountMatches and passes pointers to two different pairs of arrays.
- Use the INVOKE statement to call CountMatches and pass stack parameters and create a PROTO statement for CountMatches.
- Save and restore any registers (other than EAX) changed by your procedure.

BONUS (additional 2 points)

Find the count of all the matching elements (does not restrict to corresponding element)

Homework 3

- Basic function
  - Reports 2 matches for the following arrays
  - array1 sdword 10,5,4,-6, 2,11,12
  - array2 sdword 10,5,3, 1, 4, 2, -6

- BONUS function
  - Reports 5 matches for the following arrays
  - array1 sdword 10,5,4,-6, 2,11,12
  - array2 sdword 10,5,3, 1, 4, 2, -6

- Assuming no duplicate elements in the arrays